Vasireddy Venkatadri Institute of Technology,
Nambur, Guntur, Andhra Pradesh

Department of Computer Science and Engineering

III/II CSE - R23 Regulations

BIG DATA ANALYTICS

UNIT II - NO SQL DATA MANAGEMENT

Introduction to NoSQL – aggregate data models – key-value and document data models – relationships – graph databases – schemaless databases – materialized views – distribution models – master-slave replication – consistency - Cassandra – Cassandra data model – Cassandra examples – Cassandra clients

2.1 INTRODUCTION TO NOSQL

2.2 AGGREGATE DATA MODELS

2.3 KEY-VALUE AND DOCUMENT DATA MODELS

2.4 RELATIONSHIPS

– GRAPH DATABASES – SCHEMALESS DATABASES – MATERIALIZED VIEWS – DISTRIBUTION MODELS – MASTER-SLAVE REPLICATION – CONSISTENCY - CASSANDRA – CASSANDRA DATA MODEL – CASSANDRA EXAMPLES – CASSANDRA CLIENTS

2.1. INTRODUCTION TO NOSQL

Traditional Relational Databases

Traditional relational databases have been a cornerstone of data storage and management for several decades. They are based on the principles of the relational model, which was introduced by Edgar F. Codd in the early 1970s. These databases organize data into structured tables with well-defined schemas, using rows and columns to represent and store data.

Key Concepts of Traditional Relational Databases:

1. Tabular Structure: Data in a relational database is stored in tables, also known as relations. Each table consists of rows (records) and columns (attributes). Each row represents a specific data entry, and each column represents a different attribute or piece of information about the data.

2. Schema: Relational databases have a predefined schema that defines the structure of the data, including the tables, columns, data types, and relationships between tables. This schema enforces data

integrity and consistency.

3. SQL (Structured Query Language): SQL is a standardized language used to query and manipulate data in relational databases. It provides commands for creating, updating, querying, and deleting data, as well as defining the structure of the database.

4. ACID Properties: Relational databases are designed to maintain the ACID properties— Atomicity, Consistency, Isolation, and Durability. These properties ensure that database transactions are reliable and maintain data integrity.

Introduction to NoSQL Databases

Traditional relational databases have been the cornerstone of data storage and management for decades. However, as the requirements of modern applications have evolved, so too have the demands placed on databases. This has led to the rise of NoSQL databases, which offer a different approach to handling data.

What is NoSQL?

NoSQL, which stands for "Not Only SQL," is a term that encompasses a diverse set of database systems designed to handle various types of data and use cases. Unlike traditional relational databases, NoSQL databases are not strictly tied to a structured tabular format. They provide flexibility, scalability, and better performance for certain types of applications.

Key Characteristics of NoSQL Databases:

1. Schema Flexibility: NoSQL databases are schema-less or have flexible schemas. This means that you don't need to define a fixed schema upfront, allowing you to store data with varying structures easily.

2. Scalability: NoSQL databases are designed to scale out horizontally, making it easier to handle large amounts of data and high traffic loads. This is in contrast to traditional databases, which often scale vertically.

3. Types of NoSQL Databases:

- Document Stores: These databases store data in flexible, semi-structured documents, usually in formats like JSON or XML. Examples include MongoDB and Couchbase.

- Key-Value Stores: These databases store data as key-value pairs, making them efficient for simple lookups and caching. Examples include Redis and Amazon DynamoDB.

- Column-Family Stores: These databases organize data into columns rather than rows and are optimized for reading and writing large volumes of data. Apache Cassandra is a popular example.

- Graph Databases: These databases are designed for storing and querying highly connected data, such as social networks or recommendation engines. Neo4j is a well-known graph database.

Use Cases for NoSQL:

NoSQL databases are particularly well-suited for scenarios where traditional relational databases might struggle:

- Big Data Applications: When dealing with massive amounts of data that require horizontal scaling and flexibility.

- Real-time Analytics: NoSQL databases are often used for quickly processing and analyzing large volumes of data in real-time.

- Web Applications: NoSQL databases can handle the varying and unpredictable data structures often encountered in web applications.

- Content Management Systems: NoSQL databases are great for storing and serving content with diverse attributes.

- IoT (Internet of Things): NoSQL databases can handle the high volume and variety of data generated by IoT devices.

What is NoSQL Database

Databases can be divided in 3 types:

1. RDBMS (Relational Database Management System)
2. OLAP (Online Analytical Processing)
3. NoSQL (recently developed database)

NoSQL Database

NoSQL Database is used to refer a non-SQL or non relational database.

It provides a mechanism for storage and retrieval of data other than tabular relations model used in relational databases. NoSQL database doesn't use tables for storing data. It is generally used to store big data and real-time web applications.

History behind the creation of NoSQL Databases

In the early 1970, Flat File Systems are used. Data were stored in flat files and the biggest problems with flat files are each company implement their own flat files and there are no standards. It is very difficult to store data in the files, retrieve data from files because there is no standard way to store data.

Then the relational database was created by E.F. Codd and these databases answered the question of having no standard way to store data. But later relational database also get a problem that it could not handle big data, due to this problem there was a need of database which can handle every types of problems then NoSQL database was developed.

Advantages of NoSQL

- It supports query language.
- It provides fast performance.

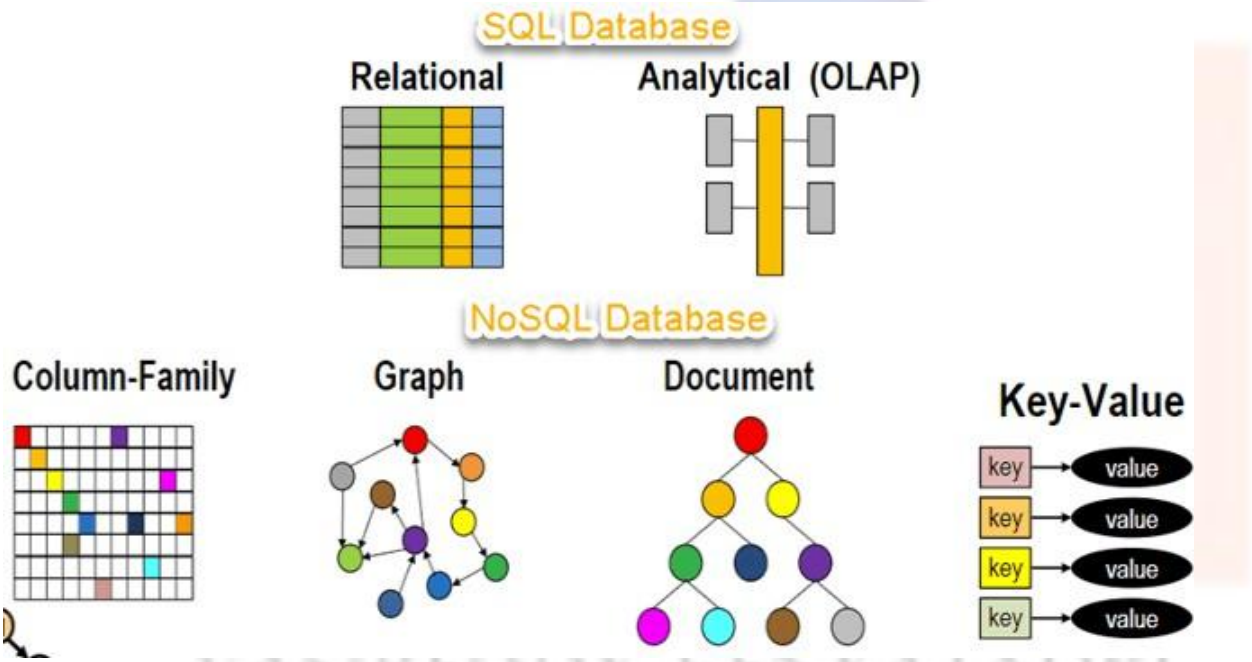
- It provides horizontal scalability.

What is NoSQL?

NoSQL Database is a non-relational Data Management System, that does not require a fixed schema. It avoids joins, and is easy to scale. The major purpose of using a NoSQL database is for distributed data stores with humongous data storage needs. NoSQL is used for Big data and real- time web apps. For example, companies like Twitter, Facebook and Google collect terabytes of user data every single day.

NoSQL database stands for "Not Only SQL" or "Not SQL." Though a better term would be "NoREL", NoSQL caught on. Carl Strozz introduced the NoSQL concept in 1998.

Traditional RDBMS uses SQL syntax to store and retrieve data for further insights. Instead, a NoSQL database system encompasses a wide range of database technologies that can store structured, semi-structured, unstructured and polymorphic data. Let's understand about NoSQL with a diagram in this NoSQL database tutorial:

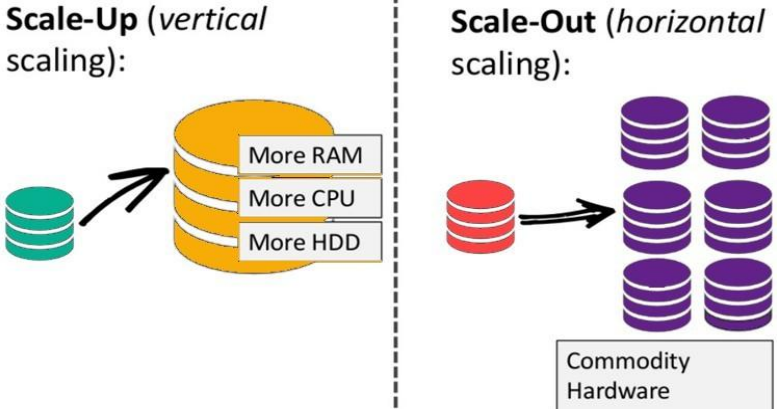


Why NoSQL?

The concept of NoSQL databases became popular with Internet giants like Google, Facebook, Amazon, etc. who deal with huge volumes of data. The system response time becomes slow when you use RDBMS for massive volumes of data.

To resolve this problem, we could "scale up" our systems by upgrading our existing hardware. This process is expensive.

The alternative for this issue is to distribute database load on multiple hosts whenever the load increases. This method is known as "scaling out."



NoSQL database is non-relational, so it scales out better than relational databases as they are designed with web applications in mind.

Brief History of NoSQL Databases

- 1998- Carlo Strozzi use the term NoSQL for his lightweight, open-source relational database
- 2000- Graph database Neo4j is launched
- 2004- Google BigTable is launched
- 2005- CouchDB is launched
- 2007- The research paper on Amazon Dynamo is released
- 2008- Facebooks open sources the Cassandra project
- 2009- The term NoSQL was reintroduced

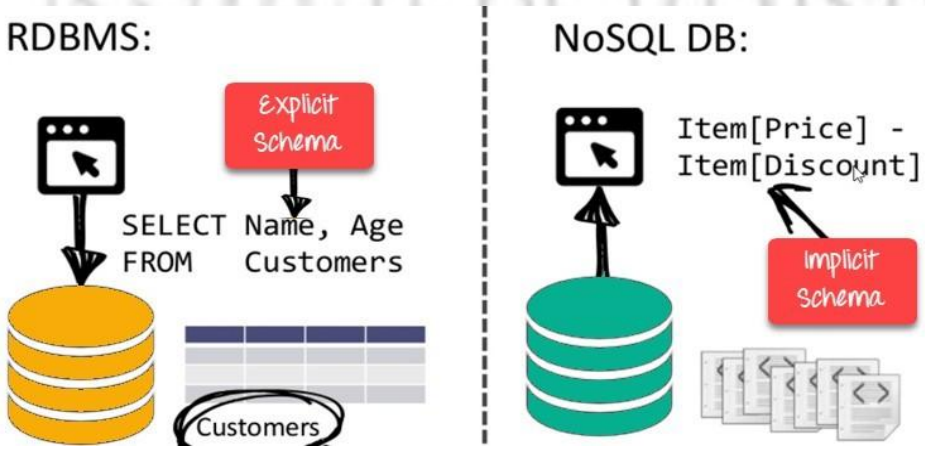
Features of NoSQL Non-

relational

- NoSQL databases never follow the relational model
- Never provide tables with flat fixed-column records
- Work with self-contained aggregates or BLOBs
- Doesn't require object-relational mapping and data normalization
- No complex features like query languages, query planners,referential integrity joins, ACID

Schema-free

- NoSQL databases are either schema-free or have relaxed schemas
- Do not require any sort of definition of the schema of the data
- Offers heterogeneous structures of data in the same domain



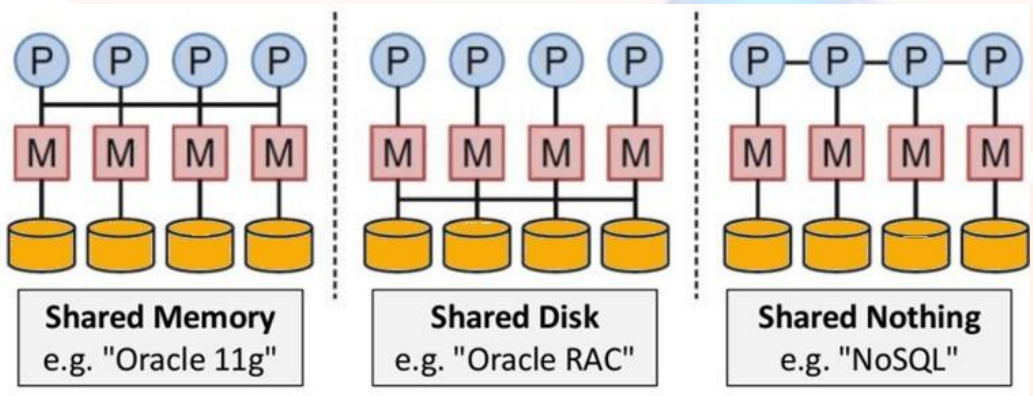
NoSQL is Schema-Free

Simple API

- Offers easy to use interfaces for storage and querying data provided
- APIs allow low-level data manipulation & selection methods
- Text-based protocols mostly used with HTTP REST with JSON
- Mostly used no standard based NoSQL query language
- Web-enabled databases running as internet-facing services

Distributed

- Multiple NoSQL databases can be executed in a distributed fashion
- Offers auto-scaling and fail-over capabilities
- Often ACID concept can be sacrificed for scalability and throughput
- Mostly no synchronous replication between distributed nodes Asynchronous Multi- Master Replication, peer-to-peer, HDFS Replication
- Only providing eventual consistency
- Shared Nothing Architecture. This enables less coordination and higher distribution.



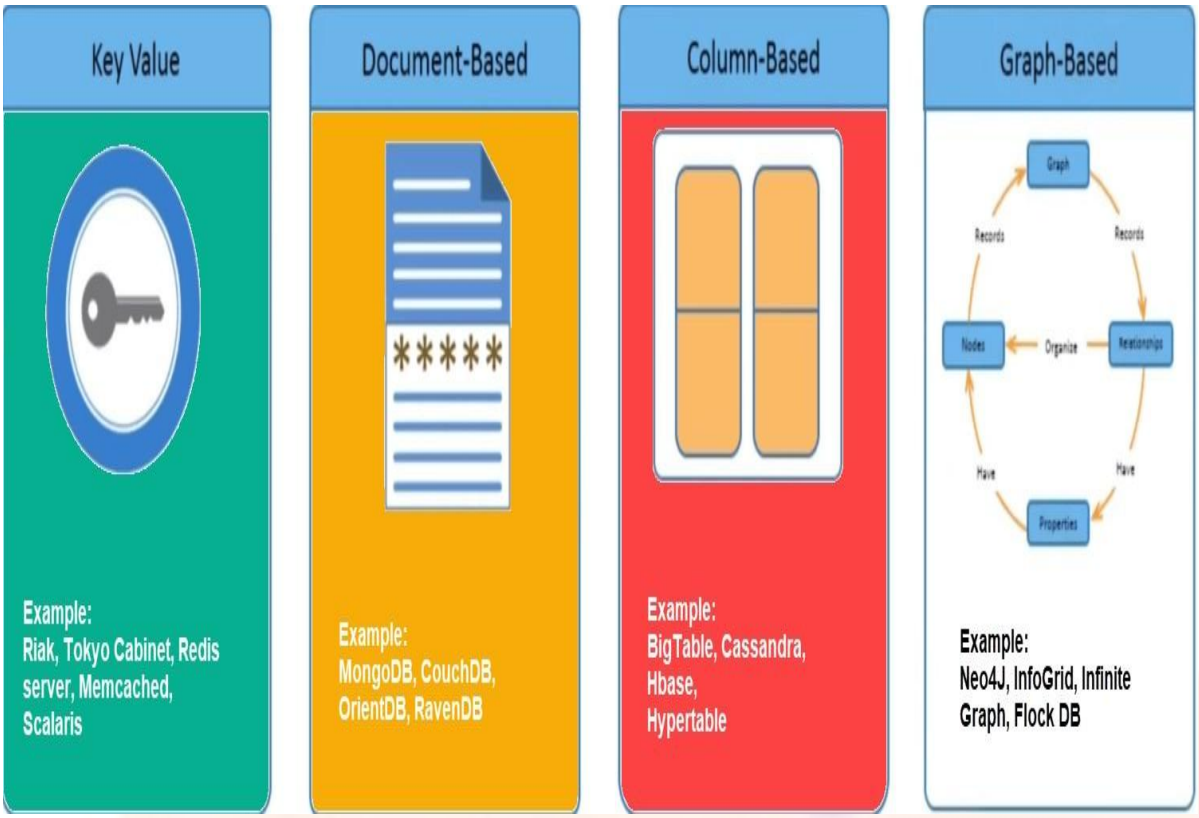
NoSQL is Shared Nothing.

Types of NoSQL Databases

NoSQL Databases are mainly categorized into four types: Key-value pair, Column-oriented, Graph-based and Document-oriented. Every category has its unique attributes and limitations. None of the above-specified database is better to solve all the problems. Users should select the database based on their product needs.

Types of NoSQL Databases:

- Key-value Pair Based
- Column-oriented Graph
- Graphs based
- Document-oriented



Key Value Pair Based

Data is stored in key/value pairs. It is designed in such a way to handle lots of data and heavy load.

Key-value pair storage databases store data as a hash table where each key is unique, and the value can be a JSON, BLOB(Binary Large Objects), string, etc.

For example, a key-value pair may contain a key like “Website” associated with a value like “Guru99”.

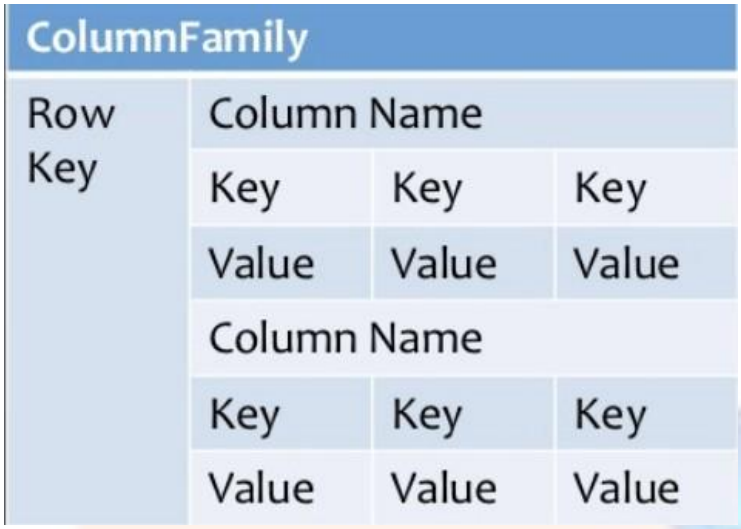
Key	Value
Name	Joe Bloggs
Age	42
Occupation	Stunt Double
Height	175cm
Weight	77kg

It is one of the most basic NoSQL database example. This kind of NoSQL database is used as a collection, dictionaries, associative arrays, etc. Key value stores help the developer to store schema-less data. They work best for shopping cart contents.

Redis, Dynamo, Riak are some NoSQL examples of key-value store DataBases. They are all based on Amazon’s Dynamo paper.

Column-based

Column-oriented databases work on columns and are based on BigTable paper by Google. Every column is treated separately. Values of single column databases are stored contiguously.



Column based NoSQL database

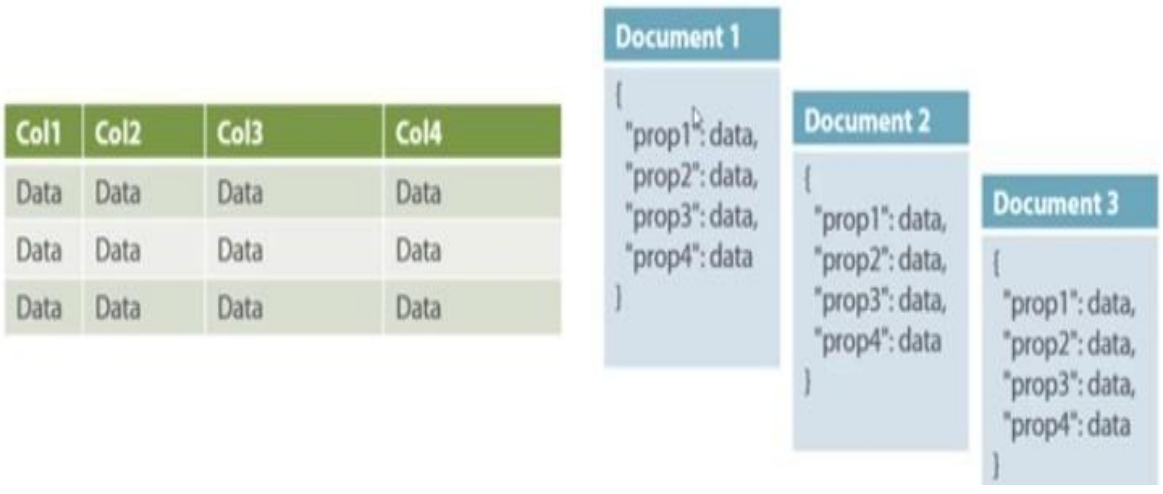
They deliver high performance on aggregation queries like SUM, COUNT, AVG, MIN etc. as the data is readily available in a column.

Column-based NoSQL databases are widely used to manage data warehouses, business intelligence, CRM, Library card catalogs,

HBase, Cassandra, HBase, Hypertable are NoSQL query examples of column based database.

Document-Oriented:

Document-Oriented NoSQL DB stores and retrieves data as a key value pair but the value part is stored as a document. The document is stored in JSON or XML formats. The value is understood by the DB and can be queried.



Relational Vs. Document

In this diagram on your left you can see we have rows and columns, and in the right, we have a

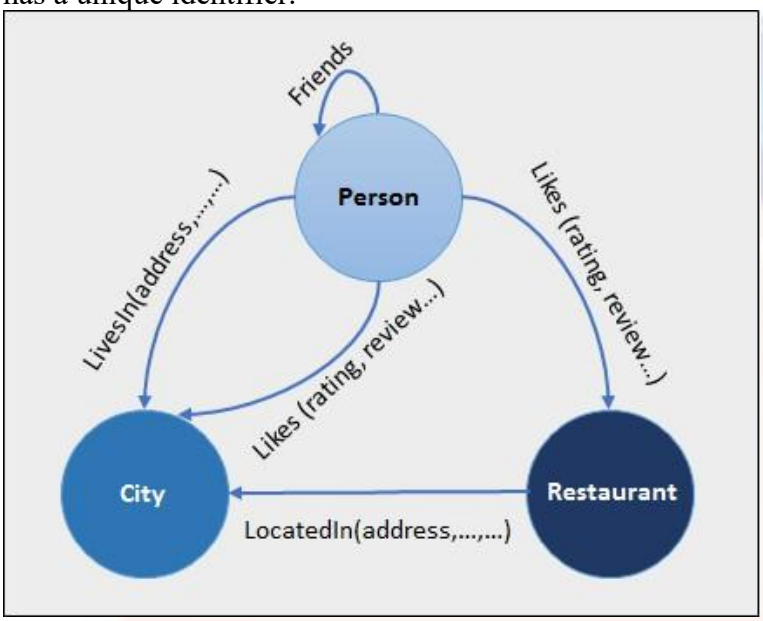
document database which has a similar structure to JSON. Now for the relational database, you have to know what columns you have and so on. However, for a document database, you have data store like JSON object. You do not require to define which make it flexible.

The document type is mostly used for CMS systems, blogging platforms, real-time analytics & e-commerce applications. It should not use for complex transactions which require multiple operations or queries against varying aggregate structures.

Amazon SimpleDB, CouchDB, MongoDB, Riak, Lotus Notes, MongoDB, are popular Document originated DBMS systems.

Graph-Based

A graph type database stores entities as well the relations amongst those entities. The entity is stored as a node with the relationship as edges. An edge gives a relationship between nodes. Every node and edge has a unique identifier.



Compared to a relational database where tables are loosely connected, a Graph database is a multi-relational in nature. Traversing relationship is fast as they are already captured into the DB, and there is no need to calculate them.

Graph base database mostly used for social networks, logistics, spatial data.

Neo4J, Infinite Graph, OrientDB, FlockDB are some popular graph-based databases.

Query Mechanism tools for NoSQL

The most common data retrieval mechanism is the REST-based retrieval of a value based on its key/ID with GET resource

Document store Database offers more difficult queries as they understand the value in a key- value pair. For example, CouchDB allows defining views with MapReduce

What is the CAP Theorem?

CAP theorem is also called brewer's theorem. It states that is impossible for a distributed data store to offer more than two out of three guarantees

1. Consistency
2. Availability
3. Partition Tolerance

Consistency:

The data should remain consistent even after the execution of an operation. This means once data is written, any future read request should contain that data. For example, after updating the order status, all the clients should be able to see the same data.

Availability:

The database should always be available and responsive. It should not have any downtime.

Partition Tolerance:

Partition Tolerance means that the system should continue to function even if the communication among the servers is not stable. For example, the servers can be partitioned into multiple groups which may not communicate with each other. Here, if part of the database is unavailable, other parts are always unaffected.

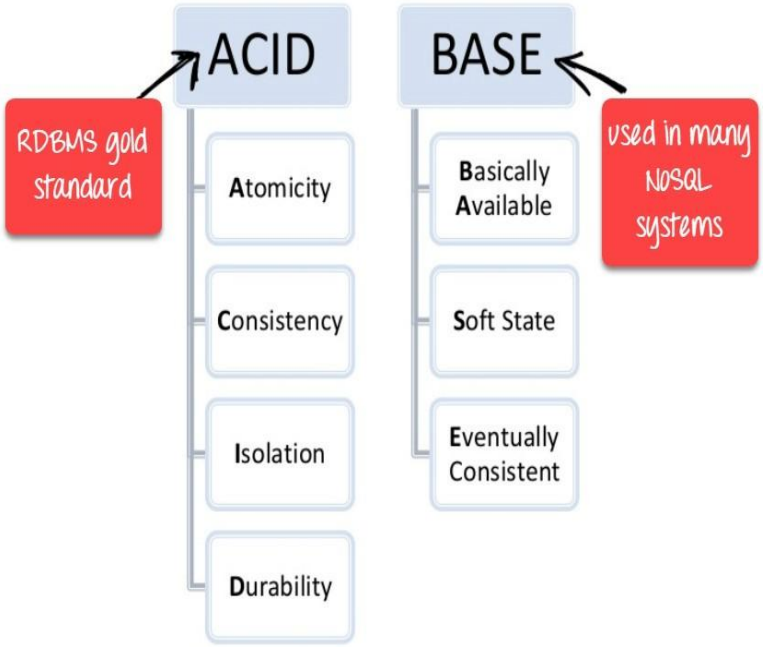
Eventual Consistency

The term "eventual consistency" means to have copies of data on multiple machines to get high availability and scalability. Thus, changes made to any data item on one machine has to be propagated to other replicas.

Data replication may not be instantaneous as some copies will be updated immediately while others in due course of time. These copies may be mutually, but in due course of time, they become consistent. Hence, the name eventual consistency.

BASE: **B**asically **A**vailable, **S**oft state, **E**ventual consistency

- Basically, available means DB is available all the time as per CAP theorem
- Soft state means even without an input; the system state may change
- Eventual consistency means that the system will become consistent over time



Advantages of NoSQL

- Can be used as Primary or Analytic Data Source
- Big Data Capability
- No Single Point of Failure
- Easy Replication
- No Need for Separate Caching Layer
- It provides fast performance and horizontal scalability.
- Can handle structured, semi-structured, and unstructured data with equal effect
- Object-oriented programming which is easy to use and flexible
- NoSQL databases don't need a dedicated high-performance server
- Support Key Developer Languages and Platforms
- Simple to implement than using RDBMS
- It can serve as the primary data source for online applications.
- Handles big data which manages data velocity, variety, volume, and complexity
- Excels at distributed database and multi-data center operations
- Eliminates the need for a specific caching layer to store data
- Offers a flexible schema design which can easily be altered without downtime or service disruption

Disadvantages of NoSQL

- No standardization rules
- Limited query capabilities
- RDBMS databases and tools are comparatively mature
- It does not offer any traditional database capabilities, like consistency when multiple transactions are performed simultaneously.
- When the volume of data increases it is difficult to maintain unique values as keys become difficult
- Doesn't work as well with relational data
- The learning curve is stiff for new developers
- Open source options so not so popular for enterprises.

Summary

- NoSQL is a non-relational DMS, that does not require a fixed schema, avoids joins, and is easy to scale
- The concept of NoSQL databases became popular with Internet giants like Google, Facebook, Amazon, etc. who deal with huge volumes of data
- In the year 1998- Carlo Strozzi use the term NoSQL for his lightweight, open-source relational database
- NoSQL databases never follow the relational model it is either schema-free or has relaxed schemas
- Four types of NoSQL Database are 1). Key-value Pair Based 2). Column-oriented Graph 3). Graphs based 4). Document-oriented
- NOSQL can handle structured, semi-structured, and unstructured data with equal effect
- CAP theorem consists of three words Consistency, Availability, and Partition Tolerance
- BASE stands for **B**asically **A**vailable, **S**oft state, **E**ventual consistency
- The term "eventual consistency" means to have copies of data on multiple machines to get high availability and scalability
- NOSQL offer limited query capabilities

2.2 AGGREGATE DATA MODELS

Aggregate data models in NoSQL databases refer to the way data is organized and stored. Instead of using fixed schemas as in relational databases, NoSQL databases adopt more flexible models that allow data to be grouped in ways that suit specific application requirements. This makes it easier to work with unstructured or semi-structured data, and also supports efficient horizontal scaling.

Example: E-commerce Product Catalog

Imagine you're building an e-commerce platform with a vast and ever-changing product catalog. In a traditional relational database, you might need to define a rigid schema for products, including fixed columns for attributes like product name, price, manufacturer, and category.

However, with a NoSQL document-oriented database, you can use an aggregate data model to handle this scenario more flexibly.

Relational Database Approach:

Product_ID	Product_Name	Price	Manufacturer	Category
1	Laptop	800	Dell	Electronics
2	T-shirt	20	Nike	Clothing
3	Book	15	Penguin	Books

In a relational database, each product requires the same set of columns, which might lead to empty or irrelevant fields for certain products.

NoSQL Document-Oriented Approach:

In a NoSQL document-oriented database, you can use a flexible schema to store products as documents, each with its own structure based on the specific product attributes. Here's how this might look:

Product 1 (Laptop):

```json
{
 "_id": 1,
 "name": "Laptop", "price": 800,
 "manufacturer": "Dell",
 "category": "Electronics",
 "specifications": {
  "screen_size": "15.6 inches",
```

```
    "processor": "Intel Core i5",
    "storage": "256GB SSD"
  }
}
```

Product 2 (T-shirt):

```json
{
  "_id": 2,
  "name": "T-shirt", "price": 20,
  "manufacturer": "Nike",
  "category": "Clothing", "size":
  "M",
  "color": "Black"
}
```

Product 3 (Book):

```json
{
  "_id": 3, "name":
  "Book", "price": 15,
  "manufacturer": "Penguin",
  "category": "Books",
  "author": "Jane Doe", "genre":
  "Fiction"
}
```

In this approach, each product is stored as a JSON document with attributes that are relevant to that specific product. Some products have additional attributes not present in others. This flexible structure allows you to accommodate different types of products without requiring a fixed schema.

Benefits:

- You can store products with varying attributes in a single collection without empty or irrelevant fields.

- When querying, you can search for specific attributes without having to navigate through columns not applicable to a particular product.
- As new types of products are introduced, you can easily adapt the document structure without changing the database schema.
- The aggregate data model supports efficient horizontal scaling as your product catalog grows.

This example illustrates how aggregate data models in NoSQL document-oriented databases provide the flexibility to organize and store data based on application requirements, allowing for efficient handling of unstructured or semi-structured data while supporting horizontal scalability.

Key-Value Data Model: The key-value data model is one of the simplest and most fundamental types of NoSQL databases. In this model, data is stored as a collection of key-value pairs, where each piece of data is associated with a unique key. The value can be of various types, including strings, numbers, or even complex structures like JSON objects. Key-value databases are highly scalable and efficient for operations involving simple read and write operations, but they may lack advanced querying capabilities compared to other NoSQL models.

****Example: User Profiles in a Key-Value Store****

Imagine you're building a social media platform, and you want to use a key-value store to efficiently manage user profiles. Each user profile contains basic information such as name, age, location, and a profile picture. The key-value data model simplifies the storage and retrieval of user profiles.

In this example, you'll use a fictional key-value database to showcase the concept.

****Key-Value Data Model:****
In a key-value store, each piece of data (the "value") is associated with a unique identifier (the "key"). Here's how user profiles might be stored:

- ****User 1:****
  - Key: `user123`
  - Value: JSON object containing user information

```json
{
  "name": "Alice", "age":
  28,
  "location": "New York",
```

```
  "profile_picture": "https://example.com/profiles/user123.jpg"
}
```

- **User 2:**
  - Key: `user456`
  - Value: JSON object containing user information
  ```json
  {
    "name": "Bob", "age":
    35,
    "location": "San Francisco",
    "profile_picture": "https://example.com/profiles/user456.jpg"
  }
  ```

- **User 3:**
  - Key: `user789`
  - Value: JSON object containing user information
  ```json
  {
    "name": "Eve", "age":
    22,
    "location": "London",
    "profile_picture": "https://example.com/profiles/user789.jpg"
  }
  ```

In this scenario, the keys (`user123`, `user456`, `user789`) are unique identifiers for each user profile, and the corresponding values are JSON objects containing user information.

Benefits:

- **Efficient Retrieval:** Retrieving a user's profile is efficient because you can directly access the data using the unique key. This is especially advantageous when the dataset is large.
- **Simple Structure:** The key-value data model's simplicity is well-suited for use cases with

straightforward data storage requirements.
- **Scalability:** Key-value stores are highly scalable and can handle a large number of simple read and write operations effectively.
- **Low Latency:** Due to the straightforward nature of key-value lookups, data retrieval is often low-latency.

Document Data Model: The document data model is another popular type of NoSQL database, designed to store and manage semi-structured or hierarchical data. In this model, data is stored in documents, which are typically formatted using JSON or BSON (binary JSON) formats. Each document can have varying structures, allowing different fields and attributes to be added without altering the entire database schema. Document databases are suitable for use cases where data is not strictly tabular and may have nested or complex relationships.

Example: Content Management System (CMS)

Consider a content management system (CMS) that needs to store and manage various types of content, including articles, images, and user comments. Each piece of content has different attributes, and the hierarchical structure can vary based on the content type. A document-oriented database is well-suited for handling this kind of semi-structured data.

Document Data Model:

In a document-oriented database, each piece of data is stored as a document. Documents are typically formatted using JSON or BSON (binary JSON), allowing for flexibility in the structure of each document.

Article Document:
- Each article is stored as a document with attributes such as title, author, content, publication date, and tags.

- Example JSON document:

```json
{
  "_id": "article123",
  "title": "Introduction to Document Databases", "author":
  "Alice",
```

"content": "Document databases offer flexible data storage...", "publication_date":
"2023-07-15",
"tags": ["NoSQL", "Databases", "Document Model"]
}
```

- **Image Document:**
  - Images are stored as documents with attributes like file name, size, format, and URL.
  - Example JSON document:
  ```json
  {
  "_id": "image456", "file_name":
  "landscape.jpg", "size": "2.5 MB",
  "format": "JPEG",
  "url": "https://example.com/images/landscape.jpg"
  }
  ```

- **Comment Document:**
  - User comments are stored as documents with attributes including commenter name, content, timestamp, and associated article ID.
  - Example JSON document:
  ```json
  {
  "_id": "comment789",
  "commenter": "Bob",
  "content": "Great article! Thanks for sharing.", "timestamp":
  "2023-07-16T08:30:00Z",
  "article_id": "article123"
  }
  ```

In this scenario, each type of content is stored as a JSON document, and the structure of each document can vary based on the content's attributes.

Benefits:

- **Flexible Schema:** The document data model allows for a dynamic and evolving schema, accommodating changes in data structure without altering the entire database.
  ```

- **Hierarchical Data:** Documents can contain nested structures, making it easy to represent complex and hierarchical data relationships.
- **Semi-Structured Data:** Document databases are well-suited for storing semi-structured or unstructured data, as well as data with varying attributes.
- **Efficient Retrieval:** Retrieving content and its associated data is efficient, as related information is often stored together within a single document.

RELATIONSHIPS

Relationships in Key-Value Data Model:

The key-value data model is simple and doesn't inherently support complex relationships between data items. However, relationships can still be established by using various strategies, often at the application level.

Example: Social Media Followers in a Key-Value Store

Imagine you're using a key-value store to manage user profiles and their followers. While a key- value store doesn't natively support relationships, you can create separate keys to represent follower relationships.

- **User 1:**
  - Key: `user123`
  - Value: JSON object with user information

- **User 2:**
  - Key: `user456`
  - Value: JSON object with user information

Follower Relationship:

```plaintext
user123-followers: [user456]
user456-followers: [user123]
```

Here, `user123-followers` and `user456-followers` keys establish a simple follower relationship. However, this approach requires manual handling and may become complex to manage at scale.

Relationships in Document Data Model:

The document data model is more flexible and directly supports complex relationships between data

items due to its ability to embed nested structures or references.

Example: Social Media Followers in a Document-Oriented Database

Using the same social media scenario, let's see how a document-oriented database handles follower relationships more naturally:

- **User 1 Document:**

```json
{
  "_id": "user123",
  "name": "Alice",
  "followers": ["user456"]
}
```

- **User 2 Document:**

```json
{
  "_id": "user456",
  "name": "Bob",
  "followers": ["user123"]
}
```

Benefits:

- **Key-Value:** While less suitable for complex relationships, relationships can still be represented using keys and values.

- **Document:** The document data model naturally supports relationships, allowing you to embed arrays or references to related data within documents.

2.5 GRAPH DATABASES:

A graph database is a type of NoSQL database designed specifically for managing highly interconnected data. It's optimized for representing and querying relationships between entities, making it suitable for scenarios where relationships are as important as the data itself. Graph databases use a graph data model, which consists of nodes (entities) and edges (relationships), allowing for efficient traversal of complex relationships.

Key Characteristics of Graph Databases:

- **Nodes and Edges:** Graph databases store data as nodes and relationships as edges, forming a network-like structure.
- **Efficient Relationships:** Graph databases excel at querying and navigating relationships, making complex queries more intuitive.
- **Schema Flexibility:** While graph databases have a schema in terms of nodes and relationships, they are more flexible compared to traditional relational databases.
- **Performance:** Graph databases are optimized for pattern matching and traversing relationships, making them efficient for graph-related queries.
- **Use Cases:** Social networks, recommendation systems, fraud detection, knowledge graphs, and any scenario with intricate relationships.

Schemaless Databases:

A schemaless database is a type of database that doesn't require a predefined schema for data storage. It's often associated with NoSQL databases, as they allow for flexible and dynamic data models. In a schemaless database, data can be added with varying structures without needing to modify the overall database schema. This approach provides agility in rapidly evolving environments where data structures may change frequently.

Key Characteristics of Schemaless Databases:

- **Dynamic Data Models:** Data can be stored without predefined tables or fixed columns, allowing for variation in data structure.
- **Flexibility:** Schemaless databases accommodate changing data requirements and evolving applications.
- **Variety of Data Types:** They can store structured, semi-structured, or unstructured data in various formats, such as JSON, BSON, XML, or key-value pairs.
- **Agility:** Schemaless databases are suitable for projects with dynamic or experimental data requirements.
- **Use Cases:** Rapid application development, projects with frequently changing data structures, scenarios with diverse data formats.